

Information Retrieval: Assignment 3

Semantic Search and Retrieval with Sentence-BERT

Prof. Toon Calders, Ewoenam Tokpo
{toon.calders, ewoenamkwaku.tokpo}@uantwerpen.be

Deadline: 12/12/2024

This project is to be executed in groups of 2 to 3 students. For further inquiries about the project, please email ewoenamkwaku.tokpo@uantwerpen.be

1 Introduction

The goal of this assignment is to explore using a *pretrained* contextual word embedding model like *Sentence-BERT (SBERT)* to build a functional semantic Search and Retrieval system. In addition to a report, each group will later prepare a presentation to present their project. More information regarding the schedule of the presentation will be given later.

2 Dataset

For this project, the dataset will be the same as that of the previous assignments. The dataset is an extract from the much larger Question Answering Dataset from MS Marco.

1. The [small dataset](#), the corresponding [small dev set of queries](#) and [result list](#) are to be used for **part 1** of the project.
2. The [large dataset](#), the corresponding [large dev set of queries](#) (*use only the first 1000 queries*) and [result list](#) (unranked) are to be used for **part 2** of the project.
3. The [test set of queries](#) should be used together with the **large dataset** for the final *Test output*. The output file for these queries must be submitted alongside your implementation (*see Project Deliverables*).

3 Assignment

So far, we have seen techniques that are effective for syntactic search. These techniques, however, struggle to capture synonymity and other semantic properties of texts. For this project, you are to implement a semantic search and retrieval system that takes a query and returns the top-k documents that are most relevant to the query. At the end of the project, you should be conversant with using contextual language models for indexing, searching, and scoring.

The project is in two parts, with the requirements for each part outlined below. Some major concepts for *part 2* will be covered in the *December 5 lecture*, so we recommend starting with *part 1* in the meantime.

3.1 Part 1: Exhaustive search and retrieval

For the first part of the assignment, the task is to implement an exhaustive semantic search and retrieval system using the ***small dataset only***, following the specifications outlined below:

1. Document embedding: Convert each document into a numerical vector representation (embedding) using a *pretrained* contextualized embedding model like *Sentence-BERT*.
2. Store all the document embeddings for search and retrieval.
3. For each query:
 - (a) Use the embedding model to generate a vector for the query.
 - (b) Compute the cosine similarity between the query vector and the vector of each document.
 - (c) Select the top 10 relevant documents ranked in descending order.
 - (d) Evaluate the results by computing the:
 - i. **Mean Precision at k** : The average precision at k of search results across all queries; for $k = 1, 3, 5, 10$. Where mean precision@k is:

$$\frac{1}{|queries|} \sum_{query} \frac{|top_k_retrieved_docs \cap relevant_docs|}{k}$$

- ii. **Mean Recall at k** : The average recall at k of search results across all queries; for $k = 1, 3, 5, 10$. Where mean recall@k is:

$$\frac{1}{|queries|} \sum_{query} \frac{|top_k_retrieved_docs \cap relevant_docs|}{|relevant_docs|}$$

3.2 Part 2: Efficient search and retrieval using Inverted indexing

For the second part of the project, the task is to implement an efficient search and retrieval system using the ***large dataset only and the first 1000 queries in its corresponding query list***. The following specifications are required for this project:

1. Document embedding: Convert each document into a numerical vector representation using a *pretrained* contextualized embedding model.
2. Implement an inverted vector index for efficient retrieval: An inverted vector indexing scheme can greatly boost the efficiency of your search algorithm. You can implement it by:
 - (a) Clustering the embeddings into k clusters.
 - (b) Computing the centroid for each cluster.
3. Document Search and Retrieval: Implement a search mechanism that retrieves relevant documents based on the query. Specifically, use the inverted index structure for retrieval. For each query:
 - (a) Use the embedding model to generate a vector representation for the query.
 - (b) Compute the cosine similarity between the *query vector* and each *cluster centroid*.
 - (c) Select the *top-k* relevant clusters and search within the documents of those clusters

4. Document ranking:
 - (a) Select the the *top-10* documents across all relevant clusters.
 - (b) Rank these documents in descending order according to their cosine similarity scores.
 - (c) Evaluation: Evaluate and discuss the results by computing the:
 - i. Compute Mean Precision at k : The average precision of search results across all queries; for $k = 1, 3, 5, 10$.
 - ii. Compute Mean Recall at k : The average recall of search results across all queries; for $k = 1, 3, 5, 10$.
5. **Extensions (for additional scores)**: You can implement one or more of the following extensions for extra scores. Beside these extensions, you are also free to implement other extensions you desire.
 - (a) **Sequence length extension**: (S)BERT has a max sequence size of 512 tokens. Implement a technique to enable you to encode full documents that have more than 512 tokens.
 - (b) **Experiment with different design choices** and hyper-parameters and discuss their impact on the system's performance.

4 Deliverables

1. **Report**: Prepare a detailed report (approximately six to eight pages) covering both parts of the project. Specifically:
 - (a) Background discussion of the various components of your system or algorithms.
 - (b) Implementation details of your system.
 - (c) Analysis and discussion of the evaluation results. Carry out a comparative evaluation of the results and the results from the previous *two assignments*. Analyze and contrast the results from the three techniques corresponding to the three assignments.
 - (d) Include a GitHub link to the implementation code in the report.

The report should be submitted as a PDF file via Blackboard. Do not submit zip files, Word documents, or bulky attachments. All code should be placed in a 'src' directory, and result files in a 'results' directory in the GitHub repository.

2. **Test output**: Test your approach on the *large dataset* with the provided *test set of queries (all queries)*.
 - (a) Return the top 10 most relevant documents (ranked in descending order of relevance) for each query.
 - (b) Save this output as *result.csv* in the format (*Query_number*, *doc_number*) provided in the example *result list* files. Include this file in the GitHub repository.
3. **Presentation**: A short PowerPoint presentation (maximum of 10 slides) to present your work. Please upload the slides to Blackboard before the day of your presentation.

A Note on Plagiarism

There is absolutely nothing wrong with using existing materials, you will even be commended for not reinventing the wheel, as long as you are not violating the copyright of other authors. Nevertheless, it is expected from you to clearly indicate whenever you used material that was not created by yourself. Clearly indicate in your submissions which parts constitute original work, which parts are taken from other works, and which parts were adapted from external sources. These sources have to be properly acknowledged in all your submissions. Concretely, this means at least the following guidelines are observed:

- Papers, books, webpages, blogs, etc. that were inspected while making the assignment will be referenced in a separate section “References”. Citations to these materials are included in the text where appropriate.
- Text fragments exceeding one sentence that are copied from other sources are clearly marked as such. You could for instance include quoted text, definitions, etc. in italics, followed by a reference. An example of how to do this: Bela Gip (2014) defines plagiarism as *“The use of ideas, concepts, words, or structures without appropriately acknowledging the source to benefit in a setting where originality is expected”*

References: (at the end of the document) Gipp, Bela. Citation-based plagiarism detection. Springer Vieweg, Wiesbaden, 2014. 57-88.

- When using code from other sources, indicate so in the report, and in the source code. This could for instance be done by adding a comment with a reference to the source of the function for each function that was copied from another source. It is recommended to include a separate folder “sources” in your GitHub repository with the original files from other authors that you used. Include source in the message of your commits.